Generation of Realistic Activity Scenarios for SUMO

Falke B. Ø. Carlsen
Department of Computer Science, Aalborg University, Aalborg, Denmark
fvejlb17@student.aau.dk

Jacob J. Rasmussen
Department of Computer Science, Aalborg University, Aalborg, Denmark
jjra17@student.aau.dk

Mathias M. Sørensen
Department of Computer Science, Aalborg University, Aalborg, Denmark
mmsa17@student.aau.dk

Nicolaj Ø. Jensen
Department of Computer Science, Aalborg University, Aalborg, Denmark
naje17g@student.aau.dk

Michele Albano
Department of Computer Science, Aalborg University, Aalborg, Denmark
mialb@cs.aau.dk

ABSTRACT

The SUMO traffic simulator is a mainstream tool that allows to model and analyse traffic and mobility scenarios. Fully realistic scenarios can be appealing for many use cases, but they require an initial large investment of resources for their creation. In fact, the usual workflow comprises the manual creation of a statistics file with detailed information about the city and its properties, up to describing how many people live and work on each road. This step is followed by the application of the tool `ACTIVITYGEN` to generate activity-based traffic. Current alternatives are based on simple randomly generated traffic, such as by means of the tool `randomTrips`. We present a compromise between the two approaches, consisting of mathematical techniques to generate schools, city gates, population density with residential and industrial areas, and a city centre. We also introduce an accompanying tool, `randomActivityGen`, which implements the approach to create `ACTIVITYGEN` statistics files automatically. Evaluation of generated scenarios shows that population and industry density, schools, and city-gates are placed realistically through testing on five representative Danish cities. The approach is also compared with the output of the tool `randomTrips` and the LuST scenario.

CCS CONCEPTS

• **General and reference** → *Evaluation*; • **Computing methodologies** → **Simulation tools**.

KEYWORDS

Simulation of Urban MObility, `ACTIVITYGEN`, Perlin noise, k-means

ACM Reference Format:
Falke B. Ø. Carlsen, Jacob J. Rasmussen, Mathias M. Sørensen, Nicolaj Ø. Jensen, and Michele Albano. 2020. Generation of Realistic Activity Scenarios for SUMO. In *MobiQuitous 2020 - 17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services (MobiQuitous '20), December 7–9, 2020, Darmstadt, Germany*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3448891.3448916

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MobiQuitous '20, December 7–9, 2020, Darmstadt, Germany
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8840-5/20/11…$15.00
https://doi.org/10.1145/3448891.3448916

1 INTRODUCTION

Realistic simulation of vehicular traffic is a valuable tool for research into traffic optimisation, to implement new vehicular technologies and Intelligent Transportation Systems, and in general to support investigation related to human mobility. Microscopic simulation involves the simulation of each traveller, and it can potentially cover all aspects of the travel, ranging from the decision regarding which transportation means to use, to when to change lane while driving.

One of the most mature microscopic simulators is SUMO [16], which can empower its users with realistic and flexible mechanisms to cover most use cases. The configuration of the scenario to be simulated involves the creation of the network topology (i.e.: a network of roads, pedestrian and bicycle ways, railways, etc) composed by *edges*, the traffic demand in term of trips (i.e.: start and destination edges for each simulated traveller, and departure time), and the *routes* (i.e.: the set of edges the traveller will pass). Activity-based demand generation is a common approach for the second step, and it involves describing the area in terms of density of population, density of workplaces, location of schools etc; a tool, for example `ACTIVITYGEN` [30], can then be used to produce the traffic demand.

The creation of realistic scenarios requires an initial large investment of resources. In fact, the usual workflow comprises the manual creation of a *statistics* file with detailed information about the city and its properties, up to describing how many people live and work on each road, which is then used by the tool `ACTIVITYGEN` to create the trips. Current alternatives are based on simple randomly generated traffic demand, such as by means of the tool `randomTrips` [14].

In this paper, we present a compromise between the two approaches, consisting in the pseudorandom auto-characterisation of a scenario. Concretely, the approach involves the generation of a statistics file from the network topology, to be used as input to the `ACTIVITYGEN` tool that produces the traffic demand. The approach utilises Perlin noise and k-means clustering to generate schools, city gates, population density with residential and industrial areas, and a city centre. We also introduce an accompanying tool,

randomActivityGen, which implements the approach to create ACTIVITYGEN statistics files automatically.

When discussing the benefits of the proposed approach, a recurring challenge was to evaluate the realism of the generated scenarios. To that aim, we tested the generated scenarios from a formal and mathematical standpoint, and by visual inspection, by comparing population and industry density, schools, and city-gates with their real-life counterparts on five representative Danish cities. The approach is also compared with the output of the tool randomTrips and the LuST scenario.

The rest of the paper is structured as follows. Section 2 discusses related work. Section 3 presents our proposed approach in terms of the approach used to generate the properties and elements of a scenario. Section 4 describes the prototype for our tool, namely randomActivityGen. In Section 5 we test our tool in terms of the realism of the generated scenarios. Finally, in Section 6 we conclude upon the quality of the approach and the tool.

2 RELATED WORK

SUMO [16] is a market leader with regards to traffic simulation, and as such has attracted a great deal of research efforts. With regards to the generation of the network topology, the two most important approaches involve either downloading a vector map of an area from a world map repository such as Open Street Map (OSM) [2] and then convert it into a SUMO network topology using a tool from the SUMO suite, or a synthetic random map can be generated, for example using the tool NETGENERATE [12]. The creation of routes from a set of trips is done by means of a *router*, and SUMO is shipped with a reasonable number of different solutions to this problem, Dynamic User Assignment router (DUAROUTER) being the most commonly used of them. Both network topology generation and the creation of routes are important tasks for the simulation of scenarios, but they are not the focus of this paper. Our proposed solution can support both real and synthetic maps, and can be used in conjunction with different routers.

Many works have addressed the problem of generating traffic demand, with solutions ranging from random generation, to the creation of full scenarios based on data collected from the real world, to activity-based demand generation.

The simplest approach to traffic demand generation is implemented by the tool randomTrips, which generates trips using a uniform probability distribution over the whole network topology, and provides many options for restricting where and how the trips are created [14]. Anyway, the creation of a good scenario is still challenging, since the user would have to run the tool multiple times, usually improving its output by trial and error, and to verify the characteristics of each output manually. Moreover, the tool does not support rush hours unless it is run multiple times with varying begin, end, and period arguments[1]. Since our proposed approach does not require much effort on the user's part, and since other tools can provide better realism than randomTrips, in section 5 we will compare our tool against the execution of this tool configured with its default parameters.

[1]See for example http://sumo-user-mailing-list.90755.n8.nabble.com/sumo-user-How-to-generate-a-Rush-hour-in-sumo-td1303.html

It is possible to use data regarding the real world to produce a realistic traffic demand. This requires manual labour and official sources on the demand which limits the size of network topologies that can be used in traffic research where realism is required. For instance, McKenney and White [17] were limited to a nine by seven city block network topology for their research into adaptive traffic signal control systems. Entirely realistic scenarios including both network topology and traffic demand were hand-crafted in the cases of the Luxembourg SUMO Traffic (LuST) [6] and Monaco SUMO Traffic (MoST) [7] scenarios, which describe the Luxembourg and Principality of Monaco greater area respectively. Bedogni et al [1] used data from the Bologna ringway to create a realistic simulation of its traffic demand. The authors of all these papers convene that imitating real traffic completely in SUMO requires a very large effort. Moreover, all these works were devoted to cover specific scenarios, thus are not readily applicable to novel scenarios as much as our proposed approach.

The tool ACTIVITYGEN can generate activity-based traffic demand by considering housing, workplaces, schools, and free-time activities [30]. However, this requires a detailed description of the cities properties, which is cumbersome to create, especially as these scenarios increase in size and complexity. In fact, authors of recent works [29] confirmed that they spent a long time to configure the simulation scenarios.

The tool SUMOpy[23] is an alternative to ACTIVITYGEN, which focuses on creating multi-modal traffic demand. Moreover, since SUMOPy's main objective is to solve the traffic assignment problem instead of being a general-purpose scenario generator, it is not particularly flexible, and it requires to write more code to customise it. An alternative approach called SAGA [5] is a valuable support that was developed while hand-crafting the MoST scenario [7], and it can be useful to produce activity-based demand. On the other hand, our proposed approach defines a scenario's statistics information, and then makes use of ACTIVITYGEN to produce the trips, thus allowing us the full power of a mainstream and mature tool. Finally, our approach is more transparent than the referred ones, since we allow the user to see what we produce: city centre, residential areas, industrial areas, city gates and schools placement.

One problem that was central to this work was how it is possible to verify if a generated scenario is realistic. Since we had access to real data, we decided to (i) use visual inspection to verify that we appear to be similar to real scenarios, and (ii) to perform statistical analysis to show that we are less "different" from real scenarios than random scenarios. This approach is well support by state-of-the-art bibliography. For example, [13] claims that its proposed method generates realistic mobility models for VANETs, and the paper justify them to be realistic by computing statistical differences between its results and real mobility traces. These differences are compared with the gap between the real traces and a random waypoint model. In the case of Bologna ringway [1], the realism of the scenario is based on the fact that the generated activity provides more traffic jams during rush hours than the ones generated by randomTrips, and by a visual inspection. The works in [11, 21], similarly to our one, use visual inspection and statistical analysis of the results collected during the simulation. In particular, the two papers focus on looking for non-uniformity of the vehicular density over time, and unrealistic traffic congestion situations respectively,

and use them as hints of the realism of the mobility models. Another example [22] compares real data with the output of a simulation, restricting the comparison to one single parameter, namely just the energy profile of electric car. Our approach to verifying our results' realism subsumes all described works, positioning itself at the same level of the state-of-the-art.

3 THE APPROACH

The existing tool ACTIVITYGEN creates activity-based traffic demand based on the data from a statistics file describing the scenario. The statistics file must specify the population, households, work hours, unemployment rate, as well as city gates, schools, and the number of people that live and work on each street. This section discusses the mathematical techniques used to find appropriate locations for the city centre, city gates, schools, as well as determining the population and industry density of the city network topology in an automated way. Aligned with the SUMO convention, the network topology is given as a graph, where nodes are junctions and edges are roads.

3.1 The city centre

The location of the city centre affects the population and industrial density. Let C denote the city centre, and be defined as the mean position of all junctions. We also define r to be the radius of the city, defined as the longest distance from C to any junction, i.e.:

$$r = \max |C - x_j|$$

where x_j is any junction's position.

3.2 City gates

City gates are entrances or exits of the city, and connect it to the surrounding world. City gates can therefore be approximated by choosing dead-ends far away from the centre since they tend to be located where the network topology is cut off from the rest of the world. To spread out city gates we generate unit vectors uniformly pointing in different directions, and then choose the dead-ends furthest in those directions.

To generate n city gates, let θ be a random number in the interval $[0, 2\pi)$, and let

$$v_i = \begin{bmatrix} \cos(\theta + \frac{2\pi i}{n}) \\ \sin(\theta + \frac{2\pi i}{n}) \end{bmatrix} \qquad \text{for } 0 < i \leq n$$

be n unit vectors uniformly pointing in different directions. Let the set D contain all dead-ends, i.e. junctions with only one neighbouring junction by road. The position x_d of the dead-end $d \in D$ is also a vector. The dot product $v_i \cdot x_d$ tells us how far d is in the direction v_i, and thus

$$g_i = \arg\max_{d \in D} v_i \cdot x_d$$

is the dead-end furthest in the direction v_i, and therefore g_i is chosen as a city gate. However, we do not allow one dead-end to be chosen for multiple directions, so once g_i has been found, g_i is removed from D.

3.3 Population and industry density

We generate the proportion of the population and workplaces per meter per street utilising Perlin noise [20], for the location of residential and industrial areas to appear as emerging characteristics of the scenario. Additionally, we consider that the population density is denser near the city centre to simulate a town centre. This density pattern is common in the real world, and even the density of industrial areas tends to follow a similar – but more decentralised – pattern [10, 31]. The formulae that we use for population and industry density are the same except for the size of some of the constants.

Let $P(x)$ be the Perlin noise sample for position x using 3 octaves and normalised to the range $[0, 1]$. Let C and r be the city's centre and radius as defined in Section 3.1. To increase contrast in the noise, to have steeper characteristics for residential and industrial areas, we will also make use of the `smoothstep` function defined as

$$\text{smoothstep}(t) = 3t^2 - 2t^3$$

Let e be an edge and x_e be the halfway point of that edge. Let $G(e) = 1 - (|C - x_e|/r)$ be the inverse normalised distance to the centre of the city relative to r. That is, $G(e) \in [0, 1]$ and $G(e)$ is close to 1 if e is close to the city centre and is close to 0 if e is far away from the city centre. Population and industry density of the edge e is then given by

$$\delta_\alpha(e) = \frac{\text{smoothstep}(P(\frac{4}{r}x_e + b_\alpha)) + G(e) \cdot w_\alpha}{1 + w_\alpha}$$

where $\alpha \in \{\text{pop}, \text{ind}\}$ are labels. The weight w_α defines how increased the density value is due to being close to the city centre. For population density $w_{\text{pop}} = 0.5$, and for industry density $w_{\text{ind}} = 0.1$. The offsets b_{pop} and b_{ind} are chosen randomly such that $b_{\text{pop}} \neq b_{\text{ind}}$. This ensures that the population and industry densities are sampled from different areas of the noise.

In essence, the formula is a noise sample added a value which size depends on the distance to the centre and then normalised. It works better for certain types of network topologies. For instance, it does not consider the shape of the network topology at all, and it is assumed that the network topology only contains one city and not too much surrounding area with country roads. It is worth noting that the higher w_α becomes the less relevant the Perlin noise becomes. From our experiments on middle-sized Danish cities, we found that $4/r$ is a scaling factor that creates a fitting district size for the cities presented in Section 5.

3.4 School placement

Schools are used in ACTIVITYGEN to model the traffic generated by the younger population. There are many different types of schools to model, and we characterise the schools as pertaining to three types: primary schools, high schools, and colleges. Each type would realistically require its own placement heuristic on a network topology. However, our approach limits the difference between school types on three attributes: student capacity, attending age, and opening times, as these are the attributes available in ACTIVITYGEN. These three attributes for the schools are chosen randomly from ranges associated with their type.

The number of schools in an area depends on the population of the city. We consider that, per 1000 inhabitants, the city has

0.2 primary schools, 0.04 high schools, and 0.015 colleges. These numbers were chosen based on the number of primary schools, high schools, and colleges in Denmark in proportion to the Danish population size [18, 26]. Therefore, the number of schools placed might only correspond to the proportion observed in Denmark.

Let us call k the total number of schools that are defined in the statistics file. To place k schools in the network topology, we first use k-means clustering [15] to partition the entire network topology into k clusters $c_1, c_2, ..., c_k$ of edges. k-means is used to form district-like clusters of edges. There are many short edges in cities, forming smaller districts, and thereby more schools pr. km^2, and fewer but longer edges outside the cities, forming larger districts, thereby fewer schools pr. km^2. This is also known as the concept of centrality [28].

For each cluster, we pick the edge that has the highest population density. Let e be an edge where both pedestrians and passenger cars are allowed (we provided this constraint not to allow schools on highways and small footpaths). Let

$$s_i = \arg\max_{e \in c_i} \delta_{\text{pop}}(e) \qquad \text{for } 0 < i \leq k$$

be the k edges where schools are placed. Choosing the edge with the highest population density, makes the schools gravitate towards areas with a high population, thereby also achieving a higher concentration of schools in these areas.

4 IMPLEMENTATION

The approach described in Section 3 is implemented in our tool, `randomActivityGen`[2]. The tool is written in Python to facilitate easy modification and integration into SUMO's existing toolchain. To interact with SUMO-entities such as network topologies and edges, the tool is built on top of the `sumolib` Python library, which is shipped as part of the SUMO software suite.

To use `randomActivityGen`, a user needs to first obtain the network topology of the city they wish to simulate. This can be obtained from multiple sources, such as OSM and through SUMO-tools [16] converted into a SUMO network topology. The user must also provide population and household information specified in an `ACTIVITYGEN` statistics file. Further entries should be given depending on the user's needs[3].

Note that `randomActivityGen` validates input statistics files and adds default values for elements, such as population brackets and work opening and closing hours, if missing. From this input `randomActivityGen` produces a populated `ACTIVITYGEN` statistics file which can be passed to `ACTIVITYGEN` to generate the scenario for SUMO. If a user wishes so, the scenario can be visualised by `randomActivityGen`, such that population weights, centre, schools, etc. are shown (see for example Fig. 1).

The tool also offers options such as specifying where the city centre lies, the number of city gates and the number of schools. If these are not given the tool will find reasonable values by itself. If some city gates are already present in the input statistics file, `randomActivityGen` will only generate the missing number of gates. This allows the user to manually place gates they know of while having the rest automatically generated. Our tool was developed and tested with version 1.3.0 and 1.5.0 from the Eclipse SUMO package.

[2]The source code, further documentation, and example scenarios are found at https://github.com/d610f20/random-activitygen-py/

[3]Documentation on `ACTIVITYGEN`'s statistics file: http://sumo.sourceforge.net/userdoc/Demand/Activity-based_Demand_Generation.html

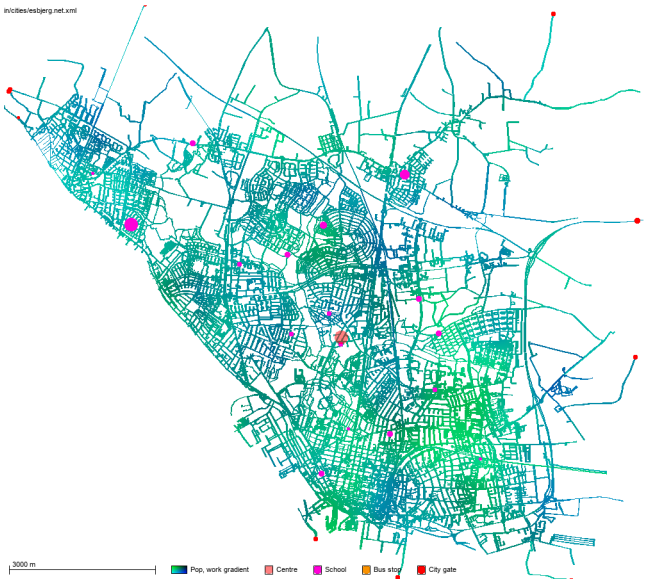


Figure 1: Visualisation of Esbjerg

City	Population	Schools	Centre (x, y)
Aalborg	117351	61	9396, 12766
Esbjerg	72037	26	7476, 1712
Randers	62482	24	19516, 6606
Slagelse	34015	16	6073, 4445
Vejen	9853	6	37800, 3790

Table 1: Testing cities and population

5 EVALUATION

The evaluation aimed at proving that our tool is a valuable trade-off between basic random demand generators such as `randomTrips`, and realistic but complex systems such as `ACTIVITYGEN`.

Danish cities of varying sizes were chosen for testing. The chosen cities are shown in Table 1 and represent both quite large Danish cities with distinct neighbourhoods and smaller, rural cities. The network topologies used for testing were acquired from OSM and census data were acquired from Statistics Denmark[4] for the year 2020 [24].

In addition to population, the number of households had to be defined. For this, the national average of 2.1 people per household was chosen to calculate an appropriate amount of households [25]. Furthermore, our tool allows for the city centre to be specified manually, and likewise for the number of schools and the number of city gates generated.

[4]dst.dk

City	Gates	Correct (#)	Correct (%)
Aalborg	40	21	52.5%
Esbjerg	33	19	57.58%
Randers	34	19	55.88%
Slagelse	28	20	71.43%
Vejen	19	14	73.68%

Table 2: Overview of the number of city gates correctly placed for each of the test cities during the test

5.1 City gate testing

To test whether the generated city gates are placed reasonably we checked whether they correctly connect the city to the outside area. We manually compared the network topologies with the original, full OSM map to determine which dead-ends were real city gates. Then for each city we found the percentage of generated city gates that matched a real city gate's location (ignoring lane direction). An overview of the results can be seen in Table 2. On the resulting percentages we performed Student's t-test. Hence we assumed the distribution of accuracy of city gates follows a normal distribution, let the null hypothesis be, that on average 50% or less of the generated city gates are not placed where a real city gate is, and let the significance level be 0.05. We found that the t-statistic was 3.1645 and the p-value was 0.01702, and we could thus reject the null hypothesis.

Looking at the percentages of correct city gates, it appears that correct placement decreases as city gate count and city population increases. A linear regression over the results outputs the line $y = 97.27 - 1.14x$ and $R^2 = 0.812$ where x is the number of city gates and y is the percentage of correctly placed gates. We assume that the main reason of the decreasing percentage is due to the fact that bigger cities have more city gates, but they also have more dead-ends that are not city gates and therefore it becomes more likely that the city gates are chosen wrongly.

5.2 School placement testing

To test school placement we manually constructed an `ACTIVITYGEN` statistics file for each of the test-cities with the location of each real school to measure the divergence between generated and real schools. Using t-tests on these divergences can then answer whether the school placement is likely to be within some maximum distance. Each generated school was assigned to a unique real school minimising the summarised distance between all assignments. This is known as the assignment problem[8] and an $O(n^3)$ implementation is found in `scipy`[5].

Since school placement is dependent on random noise, three individually seeded runs of the program were performed for each city. This shifted the focus of the test from a random instance of school placement to testing how schools were placed by the approach in general.

Assuming the divergences follow a normal distribution, we could then use a t-test to determine if the divergences were always below some bound. However, we could not know the expected divergence of our approach, so we could not simply test its performance against a known distance. Instead, we found a distance such that the t-test confirmed that divergences were smaller than that distance, by rejecting the null hypothesis:

H_0 : The generated schools are placed further than X meters away

H_1 : The generated schools are placed exactly or closer than X meters away

A one-tailed, one-sample t-test was used. If the test failed for any of the cities, we increased X by 100. We found 2100 meters to be the minimum distance, where all five cities passed at $p \leq 0.05$ at three runs per city. This means that we can accept the alternative hypothesis when $X = 2100$.

The results for each city can be seen in Table 3 and the assignments of schools in Aalborg and Esbjerg are seen in Fig. 2 and Fig. 3. The p-values and mean divergences of the different cities suggest that the school placement performs significantly better on the four smaller cities than on Aalborg.

5.2.1 KS-test. To further test the placement of schools, we used a two-sample Kolmogorov-Smirnov (KS) test [9] on the coordinates in the network topology from the generated schools and the real schools. However, the traditional KS only works in one dimension,

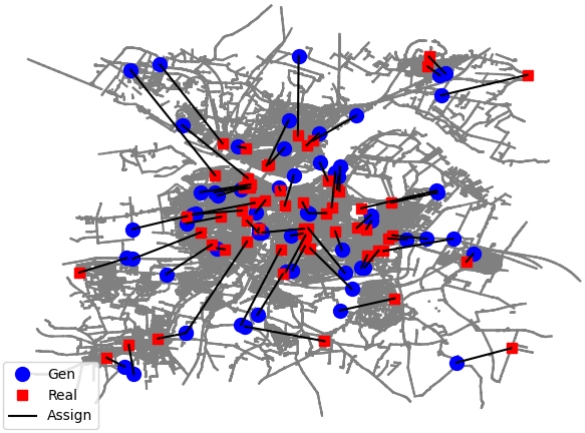


Figure 2: Assignments between real and generated schools for Aalborg

City	T-statistic	P-value	Mean divergence
Aalborg	-1.67	0.047972	1913.19 meters
Esbjerg	-9.25	1.94e−14	1257.45 meters
Randers	-8.56	7.51e−13	1247.83 meters
Slagelse	-12.99	1.80e−17	891.44 meters
Vejen	-6.29	4.03e−6	1131.56 meters

Table 3: School placement testing results on three runs of each testing city on random seeds

[5]https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html

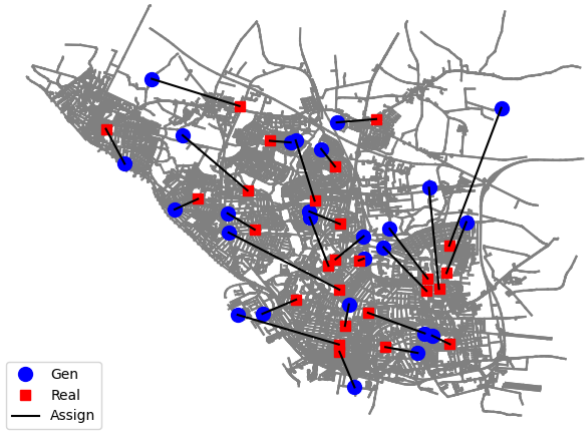


Figure 3: Assignments between real and generated schools for Esbjerg

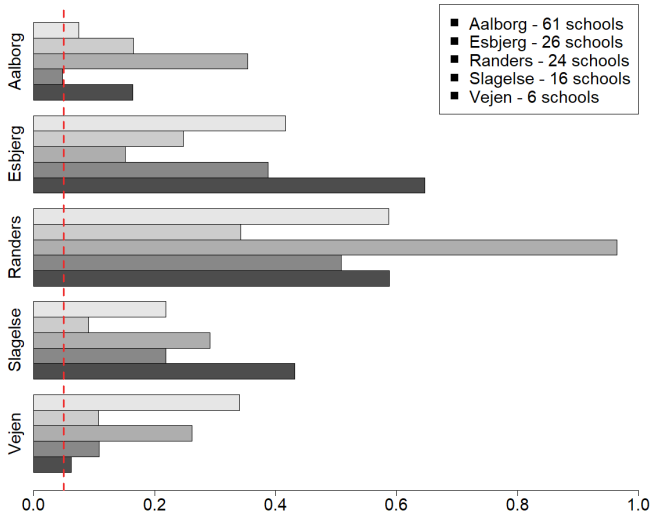


Figure 4: The p-values from five tests per city. The test is a KS between generated school positions and real school positions in the city. The red vertical line shows $p = 0.05$.

and the coordinates from the schools are in two dimensions. Therefore, we use a multidimensional version of the KS test by Fasano and Franceschini [9], that allows us to use the coordinates as sample points.

The hypotheses used in our KS test are the following[19]:

H_0 : The school locations were drawn at random from the same probability distribution

H_1 : The school locations were drawn at random from different probability distributions

The results from the KS test can be seen in Fig. 4. With a confidence level of 0.95, we fail to reject the null hypothesis on every city, thus we cannot reject that the schools are placed using the same probability distribution. One of the underlying reasons for this behaviour is that, since the generated cities are randomly generated, there could be statistical oscillations caused by outliers, and larger samples may reject the null hypothesis. Alternatively, the reason could have been that the sample size was not large enough to show the difference in the distributions. In the original paper by Peacock [19], which the multidimensional KS test we have used is based on, it is described that the sample size from either distribution should preferably be above 10. This sample size is not achieved in all tests, as Vejen only has 6 schools.

5.3 Population and industry density testing

Evaluating whether the population and industry density is realistic or not is inherently difficult to do, since there exist many feasible scenarios. Therefore, we opted to do both a visual test with visualisations and a formal test using the KS test.

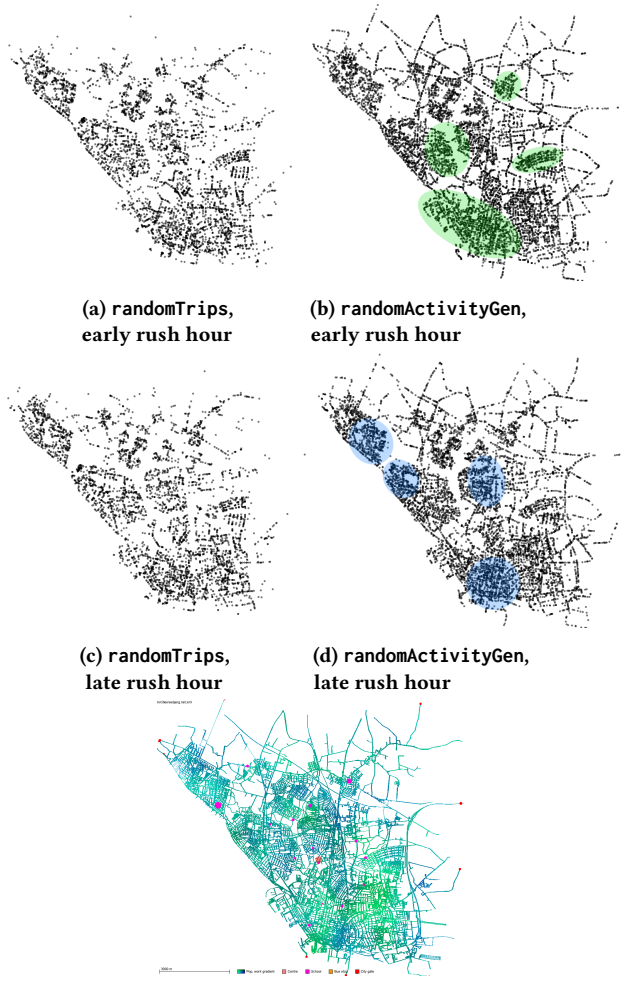
To test the population density we compared our generated trips to generated trips by `randomTrips` and trips in the LuST scenario [6]. Trips are five-dimensional data sets, as each point consists of a start and end x- and y-coordinate as well as a departure time. The population and industry density is reflected in the trips because the start and end position of a trip tell us something about where the person both lives and works, especially when looking at trips during rush hours.

Since trips are five-dimensional data points and due to the number of trips, doing a KS test with more than two dimensions is computationally infeasible [9]. Since we were testing the densities, the test was performed on the start positions. This was sufficient since trips usually start from people's homes during the early rush hour, and that trips usually start from their workplace in the late rush hour, and therefore only using start positions was enough to show both population and industry density. We defined the early rush hour as 7:00-9:00, and the late rush hour as 14:30-16:30.

5.3.1 `randomActivityGen` *vs* `randomTrips`. This KS test was done on the city of Esbjerg. The trips from `randomTrips` were generated with the following options: `-e 86400` to generate trips for the whole day, and `-p` given a fitting period that resulted in a number of trips approximately equal to the number of generated by our tool.

Let the null hypothesis be, that trips generated by the two tools originate from the same underlying probability distribution, and let the significance level be 0.05. The p-value from the test turns out to be 1.21931e−10. This very low p-value indicates that we can reject the null hypothesis, and that the trips from the two samples do not come from the same distribution. In other words, `randomActivityGen` and `randomTrips` produces different trips, and it indicates that our trips are not chosen from a uniform probability distribution as trips from `randomTrips` are.

Furthermore, visualising the trips allowed us to also visually confirm their differences. The rush hours where residential areas and industrial areas are most visible are especially interesting to inspect. The generated trips filtered by rush hours are shown on Fig. 5. The rendering shows that residential areas and industrial



(a) `randomTrips`, early rush hour

(b) `randomActivityGen`, early rush hour

(c) `randomTrips`, late rush hour

(d) `randomActivityGen`, late rush hour

(e) Population/industry noise created by `randomActivityGen`

Figure 5: Start position of trips in the city of Esbjerg as generated by `randomTrips` (a, c) and our `randomActivityGen` (b, d). Figure (e) reports population as dark green, and industry as dark blue.

areas are present and that they correspond with the Perlin noise's peaks seen in sub-figure (e). Moreover, `randomTrips` does not produce clear clusters of trips indicating, that there are no residential or industrial areas.

5.3.2 `randomActivityGen` *vs LuST.* The Luxembourg LuST scenario was created such that congestion and traffic in the city were realistic [6]. To get trips from the LuST scenario we had to transform LuST's route file into trips, which can be done with a simple script since route files contains the same information and more.

Again let the KS test's null hypothesis be that the trips by `randomActivityGen` and from LuST originates from the same probability distribution, and let the significance level be 0.05. We find that the p-value to be so low, that it is rounded to 0. As expected, the huge amount of data (215526 trips) led the KS test to reject with high confidence the hypothesis that the underlying distributions are the same. The low p-value proves that `randomActivityGen` produces a scenario with different characteristics from the LuST scenario, and further analysis is necessary to judge about `randomActivityGen` realism. Aligned with state-of-the-art approaches [1, 11, 21], visual inspection was chosen for the analysis.

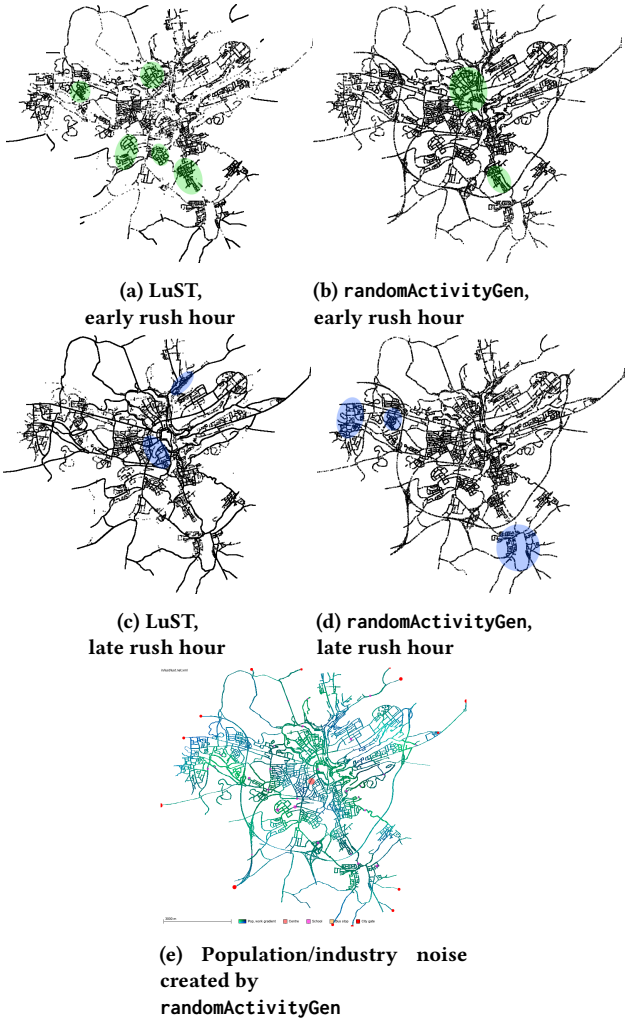
The trips of Luxembourg filtered by rush hour can be seen in Fig. 6. Note that Luxembourg has different rush hours than Denmark, which has been accommodated for based on the rush hours in LuST. The early rush hour is 7:00-9:00, and the late rush hour is 16:00-18:30. This time we see clusters of trips in both versions, proving that there are residential and industrial areas. However, they are not as prominent compared to the clusters in Esbjerg. We believe this is due to Luxembourg's small geographical size compared to its high population, which means the average density of roads is very high. The residential and industrial areas are not placed in the same locations, but that is expected, as `randomActivityGen` only guesses where the areas are based on Perlin noise. Another observation is, that during the late rush hours many of LuST's trips start on the highways, potentially indicating that a lot of traffic is going through Luxembourg according to LuST. `ACTIVITYGEN` does not generate through-going traffic [30]. This can be another reason why the p-value of the KS test is relatively low, and future work will have to cope with the issue.

5.4 Discussion on the results

From the experimental results, it appears that the approach implemented by `randomActivityGen` has best performance when the network topology contains exactly one continuous city, while it gets worse when the city branches into satellite villages. The presented approach has been tested on five Danish cities with population sizes between 10, 000 and 120, 000 people, and on Luxembourg, and it is not known how well the approach scales to bigger cities or cities that are subject to different urban plannings. Anyway, a city as big as Copenhagen would lead already to infeasible run time and memory requirements for the SUMO simulation platform, thus it is outside the scope of this work for the limitations of the target application.

The chosen five cities were primarily built before 1880, and the widespread adoption of cars has not influenced the design of the cities nor contributed to urban sprawl. The old cities' way of coping with population growth is to increase density in the established housing.[10] In [28] it is shown that most European cities exhibit similar properties, which may indicate that the encouraging results obtained on the five Danish cities can be extrapolated to other similar European cities.

With regards to US cities, and in general to cities that were planned and built more recently, it appears that different structure and scaling rules apply. For example, recent works [3, 27] affirm that the total length of the street network in modern cities scales linearly with the number of road intersections, which clashes with the sublinear growth typical of European old cities [27]. This can be related to a more rational city planning that took into account vehicular traffic. To be able to apply `randomActivityGen` in those scenarios, it will be necessary to add a parameter to specify which



(a) LuST, early rush hour

(b) randomActivityGen, early rush hour

(c) LuST, late rush hour

(d) randomActivityGen, late rush hour

(e) Population/industry noise created by randomActivityGen

Figure 6: Start position of trips in the city of Luxembourg as created in LuST (a, c) and our randomActivityGen (b, d). Figure (e) reports population as dark green, and industry as dark blue.

style (old vs modern) of planning is behind the city being modelled by randomActivityGen.

It has been a reoccurring challenge to prove that a scenario and its features are realistic from a formal and mathematical standpoint. For instance, we used a KS test to compare our generated population density in Luxembourg to the LuST scenario. We found that some characteristics do not match the output of randomActivityGen but still the generated scenarios share similar patterns. Other characteristics, and in particular the statistics of the generated traffic during late rush hours, appear to differ from LuST's since the latter generates most trips starting positions on highways to simulate traffic is going through Luxembourg. Covering this use case implies adding a parameter to randomActivityGen to require to generate a percentage of the traffic on the highways instead than on industrial areas.

6 CONCLUSION

In this paper we presented a mathematical approach for generating the city elements for ACTIVITYGEN's statistics files allowing users of SUMO to easily and quickly generate realistic, activity-based traffic demand. Our proposed approach, and its accompanying tool randomActivityGen, allows rapid generation of realistic scenarios that may reduce the time needed for preparing investigation efforts regarding vehicular mobility, which is a fundamental building block to simulate Intelligent Transportation Systems. The main benefit of using the presented approach and randomActivityGen is that it taps into the activity-based traffic modelling of ACTIVITYGEN, which considers work commuting, rush hours, free time activities, as well as the location of the city centre, residential areas, industrial areas, schools and city gates. The approach allows to harvest all the benefits of ACTIVITYGEN. Compared to randomTrips, randomActivityGen is almost as easy to use – since it just adds one more step to the workflow to set up a simulation – but creates traffic with more details and structure, such as residential areas and industrial areas, thus with better realism.

We used network topologies of various Danish cities in the evaluation of the approach. The evaluation showed that our approach does not replicate reality, but it can capture some of their statistical characteristics. It was shown that at least 50% of the city gates are placed close to where a real city gate is located. However, it was also seen that the bigger the city is, the fewer city gates are placed correctly.It was also evaluated how far generated schools' location diverges from the real locations of schools in the test cities, and in particular each generated schools was always placed within 2100 meters of a real-world school. Additionally, a KS test showed that we cannot reject that the generated and the real school locations are generated from the same statistical distributions, indicating the schools are placed realistically. Regarding population and industrial density, our KS testing showed that the generated densities are different from the densities generated by randomTrips, and a visual inspection hinted that randomActivityGen densities present patterns more similar to what can be expected in the real world.

Based on our evaluations and other limitations discussed in Section 5, we assess that our tool can be used instead of randomTrips when the network topology contains one city of reasonable size that is not composed by smaller villages. Our tool adds an extra step to the pipeline and requires ACTIVITYGEN to produce the trips for the simulation, but the resulting traffic is more realistic than the traffic generated by randomTrips.

One of the limitations of our tool is that it does not consider the use case where most traffic is just going through the city instead of being originated and ended in the city. To cover the use case, we plan to allow the user to set up a percentage of the traffic that must be either generated or ended on the highways, instead than on the city's industrial areas.

As a future work, we aim to support bus stops and bus routes generation.A potential solution can be placing bus stops using the principle of Poisson-Disc sampling[4] yielding evenly spaced bus-stops. Normally Euclidean distance is used, but two close points in Euclidean distance may not have a short route between them. Using the shortest path as an alternative distance metric may yield good results, but having to find the shortest path whenever a point

is placed increases the run time complexity by a long shot. We also experimented with using k-means clustering to choose edges with bus stops. This approach is similar to the presented approach that places schools using k-means clustering. The experiments were promising both in terms of results and run time. However, for ACTIVITYGEN, bus stops cannot be used except together with bus routes. For bus routes, we propose the use of either random walks or a heuristic metric. Finding optimal routes seems to be an NP-hard problem similar to the travelling salesman problem so an approximation algorithm is appropriate.

An ongoing effort aims at integrating the tool into the SUMO software suite, to be able to harvest its benefit when simulating an Intelligent Transportation System.

ACKNOWLEDGMENTS

The research presented in this paper is supported in part by the DiCyPS (Center for Data-Intensive Cyber-Physical Systems) project of the Innovation Fund Denmark.

REFERENCES

[1] Luca Bedogni, Marco Gramaglia, Andrea Vesco, Marco Fiore, Jérôme Härri, and Francesco Ferrero. 2015. The Bologna Ringway dataset: improving road network conversion in SUMO and validating urban mobility via navigation services. *IEEE Transactions on Vehicular Technology* 64, 12 (2015), 5464–5476.

[2] Jonathan Bennett. 2010. *OpenStreetMap*. Packt Publishing Ltd, Birmingham, UK.

[3] Geoff Boeing. 2020. A multi-scale analysis of 27,000 urban street networks: Every US city, town, urbanized area, and Zillow neighborhood. *Environment and Planning B: Urban Analytics and City Science* 47, 4 (2020), 590–608. https://doi.org/10.1177/2399808318784595

[4] Robert Bridson. 2007. Fast Poisson disk sampling in arbitrary dimensions. In *ACM SIGGRAPH 2007 sketches (SIGGRAPH '07)*. ACM, San Diego, California, USA, 22–es.

[5] Lara Codecá, Jakob Erdmann, Vinny Cahill, and Jérôme Harri. 2020. SAGA: An Activity-based Multi-modal Mobility Scenario Generator for SUMO. In *SUMO 2020, SUMO User Conference, From Traffic Flow to Mobility Modeling, October 26-28, 2020, Cyberspace*. DLR, Berlin, Online, 20 pages.

[6] Lara Codecá, Raphaël Frank, Sébastien Faye, and Thomas Engel. 2017. Luxembourg SUMO Traffic (LuST) Scenario: Traffic Demand Evaluation. *IEEE Intelligent Transportation Systems Magazine* 9, 2 (2017), 52–63.

[7] Lara Codecá and Jérôme Härri. 2018. Monaco SUMO Traffic (MoST) Scenario: A 3D Mobility Scenario for Cooperative ITS. In *SUMO 2018, SUMO User Conference, Simulating Autonomous and Intermodal Transport Systems, May 14-16, 2018, Berlin, Germany*. DLR, Berlin, Berlin, GERMANY, 43–55.

[8] David F Crouse. 2016. On implementing 2D rectangular assignment algorithms. *IEEE Trans. Aerospace Electron. Systems* 52, 4 (2016), 1679–1696.

[9] G. Fasano and A. Franceschini. 1987. A multidimensional version of the Kolmogorov–Smirnov test. *Monthly Notices of the Royal Astronomical Society* 225, 1 (03 1987), 155–170. https://doi.org/10.1093/mnras/225.1.155 arXiv:https://academic.oup.com/mnras/article-pdf/225/1/155/18522274/mnras225-0155.pdf

[10] Avery Guest. 1973. Urban growth and population densities. *Demography* 10, 1 (1973), 53–69.

[11] Jérôme Härri, Fethi Filali, Christian Bonnet, and Marco Fiore. 2006. VanetMobiSim: generating realistic mobility patterns for VANETs. In *Proceedings of the 3rd international workshop on Vehicular ad hoc networks*. ACM, Los Angeles, CA, USA, 96–97.

[12] Markus Hartinger, Daniel Krajzewicz, Jakob Erdmann, and Michael Behrisch. 2001. NETGENERATE documentation. https://sumo.dlr.de/docs/NETGENERATE.html.

[13] Feliz Kristianto Karnadi, Zhi Hai Mo, and Kun-chan Lan. 2007. Rapid generation of realistic mobility models for VANET. In *2007 IEEE wireless communications and networking conference*. IEEE, Hong Kong, 2506–2511.

[14] Daniel Krajzewicz, Jakob Erdmann, and Michael Behrisch. 2020. SUMO randomTrips.py documentation. https://sumo.dlr.de/docs/Tools/Trip.html.

[15] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.

[16] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie WieBner. 2018. Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, Maui, Hawaii, USA, 2575–2582.

[17] Dave Mckenney and Tony White. 2013. Distributed and adaptive traffic signal control within a realistic traffic simulation. *Engineering Applications of Artificial Intelligence* 26, 1 (2013), 574–583.

[18] Børne og undervisningsministeriet styrelsen for IT og læring. 2020. Dan udtræk af aktive institutioner fra Institutionsregisteret. https://statistik.uni-c.dk/instregudtraek/.

[19] J. A. Peacock. 1983. Two-dimensional goodness-of-fit testing in astronomy. *Monthly Notices of the Royal Astronomical Society* 202, 3 (03 1983), 615–627. https://doi.org/10.1093/mnras/202.3.615 arXiv:https://academic.oup.com/mnras/article-pdf/202/3/615/2902320/mnras202-0615.pdf

[20] Ken Perlin. 1985. An Image Synthesizer. *SIGGRAPH Comput. Graph.* 19, 3 (July 1985), 287–296. https://doi.org/10.1145/325165.325247

[21] Marco Rapelli, Claudio Casetti, and Giandomenico Gagliardi. 2019. TuST: from Raw Data to Vehicular Traffic Simulation in Turin. In *2019 IEEE/ACM 23rd International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*. IEEE, Cosenza, Italy, 1–8.

[22] Insaf Sagaama, Amine Kchiche, Wassim Trojet, and Farouk Kamoun. 2019. Evaluation of the Energy Consumption Model Performance for Electric Vehicles in SUMO. In *2019 IEEE/ACM 23rd International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*. IEEE, Cosenza, Italy, 1–8.

[23] Joerg Schweizer, Federico Rupi, Francesco Filippi, and Cristian Poliziani. 2018. Generating activity based, multi-modal travel demand for SUMO. *EPiC Series in Engineering* 2 (2018), 118–133.

[24] Danmarks Statistik. 2020. BY1: Population 1. January by urban, rural areas, age and sex. https://www.statistikbanken.dk/statbank5a/SelectVarVal/Define.asp?Maintable=BY1.

[25] Danmarks Statistik. 2020. Dwellings by unit, region, use and time. https://www.statistikbanken.dk/bol106.

[26] Danmarks Statistik. 2020. Folketal. https://www.dst.dk/da/Statistik/emner/befolkning-og-valg/befolkning-og-befolkningsfremskrivning/folketal.

[27] Emanuele Strano, Andrea Giometto, Saray Shai, Enrico Bertuzzo, Peter J Mucha, and Andrea Rinaldo. 2017. The scaling structure of the global road network. *Royal Society open science* 4, 10 (2017), 170590. https://doi.org/10.1098/rsos.170590

[28] Emanuele Strano, Matheus Viana, Luciano da Fontoura Costa, Alessio Cardillo, Sergio Porta, and Vito Latora. 2013. Urban street networks, a comparative analysis of ten European cities. *Environment and Planning B: Planning and Design* 40, 6 (2013), 1071–1086.

[29] Luis Urquiza-Aguiar, William Coloma-Gómez, Pablo Barbecho Bautista, and Xavier Calderón-Hinojosa. 2020. Comparison of SUMO's vehicular demand generators in vehicular communications via graph-theory metrics. *Ad Hoc Networks* 106 (2020), 102217.

[30] Piotr Woznica, Walter Bamberger, Daniel Krajzewicz, Jakob Erdmann, and Michael Behrisch. 2001. ACTIVITYGEN documentation. https://sumo.dlr.de/docs/ACTIVITYGEN.html

[31] Richard Saul Wurman. 2020. Urban Observatory. https://www.urbanobservatory.org/.